

exp-fips203-mlkem-pke-decrypt-k3

FIPS 203 Alg.15 ML-KEM-768 PKE Decrypt 实现方案

ascendc 工程 (ML-KEM-768 W4a / E15)

2026-07-26

摘要

本文档描述 `examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-decrypt-k3/` 在 Atlas A2 / Ascend910B4 上以 AscendC 实现 FIPS 203 Algorithm 15, 即 ML-KEM-768 ($k=3$) PKE Decrypt。

参数锁定: 几何、I/O 与 launch 全部锁定到 `docs/specs/fips203-mlkem768-parameter-card.md` §3.2 D15。**行为基线:** 活跃探针 `ascendc-tests/ml-kem/ml-kem-768/pass-fix-f203-alg15-pke-decrypt-device-k3/`。**生产 I/O:** input/ 为 `dk_pke.bin` (1152B)、`c.bin` (1088B) 与静态 LUT; output/ 仅 `m.bin` (32B)。**计划 AI Core 数:** 1 launch; `f203_decrypt_device_fused` 为 MIX blockDim=1 (1 AIC + 2 AIV)。**禁止:** 创建 `examples/stable/ml-kem/ml-kem-768/`; 从任何 `frozen/` 目录复制、改写或移植源码; 用 k4 的 `dk=1536`、`c=1568`、`du/dv=11/5` 或 `4/8` 路假 poly 替代 k3 几何。

目录

1	工程边界	2
1.1	允许的代码来源与依赖	2
1.2	禁止项	2
2	数学语义 (Alg.15, $k=3$, $n=256$, $q=3329$)	2
2.1	输入输出	2
2.2	阶段公式	2
3	AscendC 实现规划	3
3.1	Launch 与 AI Core 规模	3
3.2	GM 几何	3
3.3	核内同步	3
4	AscendC API 表	4
4.1	评估过但未采用	5
5	数学步骤覆盖率	5

目录	2
6 验证计划	5
6.1 默认验收	5
6.2 可选对照	6
7 产出物	6

1 工程边界

1.1 允许的代码来源与依赖

来源	用途
ascendc-tests/ml-kem/m	活跃 D15 k3 行为基线；可复制后自包含适配到本
l-kem-768/pass-fix-f20	exp, 最终不得保留对探针目录的编译期
3-alg15-pke-decrypt-de	include/import
vice-k3/	
examples/incubating/ml	活跃 k4 incubating 结构参考；仅 retarget 到 k3 锁
-kem/ml-kem-1024/exp-f	定几何与字节长
ips203-mlkem-pke-decry	
pt-k4/	
library/shared/	公共设备侧头、统一整数压缩/解压辅助等共享库
CANN / tikicpulib /	编译、CPU 孪生与 SIM 运行环境
CAModel	

1.2 禁止项

- 禁止从 ascendc-tests/frozen/ 或 examples/frozen/ 拿源码、customspec 或路线。
- 禁止将 u' 、 v' 、 $\hat{s}$ 、 $\hat{u}$ 、 $\hat{w}$ 、 w 等中间态作为默认 I/O 落盘；debug dump 只能显式开关启用。
- 禁止把 k4 的 $dk=1536$ 、 $c1=1408/c2=160$ 、 $du=11/dv=5$ 、4/8 路 pad 当作默认。
- 禁止新建或写入 stable-768；本目录只作为 incubating 预研实现。

2 数学语义 (Alg.15, $k = 3$, $n = 256$, $q = 3329$)

2.1 输入输出

张量 / 文件	契约
input/dk_pke.bin	1152B; $\hat{s}$ 的 ByteEncode_{12} , 即 $3 \times 384\text{B}$
input/c.bin	1088B; $c = c_1 \ c_2$, 其中 $c_1 = 960\text{B}$ ($d_u = 10$, 3 poly), $c_2 = 128\text{B}$ ($d_v = 4$, 1 poly)
input/lut_even_stacked.bin	Not a NTT Stage2 LUT; 由本目录 golden 脚本生成; 不是
lut_odd_stacked.bin	用户可见交付输入
output/m.bin	32B, $\text{ByteEncode}_1(\text{Compress}_1(w))$, 为唯一产物

2.2 阶段公式

1. 行 1-2: 拆分 $c_1 = c[0 : 960]$, $c_2 = c[960 : 1088]$ 。

2. 行 3: $u' \leftarrow \text{Decompress}_{10}(\text{ByteDecode}_{10}(c_1))$, 得到 3 个 poly。
3. 行 4: $v' \leftarrow \text{Decompress}_4(\text{ByteDecode}_4(c_2))$, 得到 1 个 poly。
4. 行 5: $\hat{s} \leftarrow \text{ByteDecode}_{12}(\text{dk_pke})$, 得到 3 个 NTT 域 poly。
5. 行 6: $\hat{u} \leftarrow \text{NTT}(u')$; $\hat{w} \leftarrow \sum_{j=0}^2 \text{MultiplyNTTs}(\hat{s}[j], \hat{u}[j])$; 再对 $\hat{w}$ pad 到 k3 前缀后执行 INTT, 得到 w_t 。
6. 行 7: $w = (v' - w_t) \bmod q$, $m = \text{ByteEncode}_1(\text{Compress}_1(w))$ 。

尾段方向锁定: Decrypt 行 7 是 $\text{ByteEncode}_1(\text{Compress}_1(w))$, 不是 Encrypt 行 20 的 $\text{Decompress}_1(\text{ByteDecod}$

3 AscendC 实现规划

3.1 Launch 与 AI Core 规模

Launch	核类型 / blockDim	数据划分与理由
1 fused	MIX, blockDim=1	1 AIC + 2 AIV; AIV0 负责 unpack、 $\hat{s} \cdot \hat{u}$ 与尾 pack 的关键串行段; NTT/INTT 使用真实 k3 分片, AIV0 处理 poly0/1, AIV1 处理 poly2。选择单 MIX launch 是参数卡 §3.2 D15 锁定值, 继承 D15 探针已验 GATE 4/8 与 softSyncGm。

3.2 GM 几何

对象	锁定几何
u'	[3,256] int32; 由 c1 d10 unpack + Decompress10 得到
v'	[1,256] int32; 由 c2 d4 unpack + Decompress4 得到
$\hat{s}$	[3,256] int32; 由 dk_pke ByteDecode12 得到
$\hat{u}$ NTT	真 polyvec3; AIV 2+1, 禁止读写第 4 个假 poly
$\hat{w}$ / INTT	$\hat{w}$ 写入 k3 前缀, 后续 INTT 只消费第 0 个时域 poly; Stage123 内不引入 k4 batch
Tail pack	Compress1 产 256 bit; ByteEncode1 LSB-first pack 为 32B

3.3 核内同步

单 kernel 内部沿用活跃 D15 k3 已验 FSM: NTT 使用 CrossCore flag 1/2/3; su_dot 后通过 softSyncGm 汇合双 AIV; 段间用 GATE 4→8 分隔 NTT 与 INTT; INTT 使用 flag 1/3, 避免复用 NTT 的 flag 2。CPU 仅作为逻辑孪生; SIM 是同步与搬运验收门槛。

4 AscendC API 表

本实现不引入 E15 独有的新 AscendC API；下表 API 均已有索引记录，使用约束按 `library/documents/CANN-AscendC算子开发接口参考-查阅索引.md` 执行。

API 名	分类 / 文档	Atlas A2	本用例 dtype	备注
GetBlockIdx / GetSubBlockIdx	系统变量 / 资源	√	标量索引	区分 AIV/AIC 子核与串行 owner；禁止把 CPU 打印误认为多核性能
GlobalTensor / LocalTensor	基础结构	√	uint8/int8/int32	GM/UB 张量契约必须与本 spec 几何一致
DataCopy / Duplicate	Memory / 初始化	√	uint8/int8/int32	禁止 $\hat{w}$ 、清 workspace、GM/UB 交接；禁止标量写 GM 再 MTE 读
PipeBarrier	Pipe 同步	√	—	MTE/Vector/Cube 阶段交界显式同步
CrossCoreSetFlag / CrossCoreWaitFlag	MIX 同步	√	—	NTT/INTT AIV/AIC 通过 GM 交接时使用；CPU 过不能删
Mmad / Fixpipe	矩阵计算	√	int8*int8->int32	882 MMAD；逻辑 k=3，硬件 M 对齐垫不是假 poly
ShiftRight	矢量算术	√	int32	Decompress/Compress 与 Barrett 辅助右移
Muls / Add / Sub	矢量算术	√	int32/int16	Decompress、 $v' - w$ 、Compress1 辅助
Cast	类型转换	√ (受限)	int32/int16/int8	遵守索引中 A2 Cast 链约束；不得假设存在 int32->int8 单跳

GetValue / SetValue	LocalTensor 标量访问	√	uint8/int32 仅用于 ByteEncode1 bit pack 等标量缺口
---------------------	------------------	---	--

4.1 评估过但未采用

API / 路线	不采用原因
真向量 ByteEncode1 bit pack	d=1 跨字节 bit 流，已有标量 pack 更简单且 D15 已验；本轮不把尾段作为性能变量
高阶 Matmul<>	本路线继承 D15 手写 MMAD + 平面 mat_c 契约；高阶 Matmul 输出布局与 MIX 交接不作为本轮变量
Gather 用于 NTT S1-S3	参数卡与 ML-KEM NTT 定稿禁止；NTT 三段保持平面 mat_c + 连续 poly 批处理

5 数学步骤覆盖率

数学步骤	实现方式 / API	覆盖	缺口说明
ByteDecode/Decompress d10/d4	标量 unpack + 统一整数向量 Decompress	部分向量	bit unpack 标量缺口沿用 D15 已验实现
ByteDecode12($\hat{s}$)	标量 unpack 到 GM/UB	部分向量	与 D15 基线一致；不引入新向量 unpack
NTT(u') 与 $\hat{s} \cdot \hat{u}$	MIX AIV/AIC + Mmad + Alg.11	100% 设备	NTT 内无 Gather；su_dot 串行 owner 明确
INTT($\hat{w}$)	MIX AIV/AIC + Stage3 merge	100% 设备	pad 到 k3 前缀，不读写假第 4 poly
Compress1 + ByteEncode1	Compress1 向量 Barrett + 标量 bit pack	部分向量	ByteEncode1 bit pack 标量缺口沿用 D15 已验实现

6 验证计划

6.1 默认验收

```
cd examples/incubating/ml-kem/ml-kem-768/exp-fips203-mlkem-pke-decrypt-k3
bash run.sh -r cpu -v Ascend910B4
```

```
SIM_DIRECT=1 bash run.sh -r sim -v Ascend910B4
```

期望: m.bin 为 32B; 与 golden 对拍 [verify] PASS max=0 (32 bytes)。SIM 通过后记录 Total tick 到 qa/active_sim_regress_summary.md, 并检查用例根目录无 stray core*.dump、profile_*log*.toml。

6.2 可选对照

后续可补 D14→D15 PKE round-trip, 但 E15 本轮门禁为 CPU + SIM_DIRECT=1 sim。liboqs 仅作为黑盒 I/O oracle; 禁止把 liboqs 源码实现同构移植进 AscendC。

7 产出物

- exp-fips203-mlkem-pke-decrypt-k3-实现方案-customspec.tex
- exp-fips203-mlkem-pke-decrypt-k3-实现方案-customspec.pdf
- 后续实现文件须与本 customspec 同目录自包含, 且自研 Python/C/AscendC 写详细中文注释。